Event-Driven Architecture

with Node.js, TypeScript, Kafka and Temporal

Concepts, patterns and a complete, tested order platform: typed event contracts, idempotent producers, consumer groups, the transactional outbox, idempotent consumers, dead-letter topics, choreography and orchestration, a Temporal saga driven by Kafka, CQRS read models, failure drills and production practices.

Verified with Kafka 4.1 (KRaft) · @confluentinc/kafka-javascript 1.11 · @temporalio/* 1.24 · Node.js 24 · TypeScript 7

Contents

Part I — Concepts 3

1. What Event-Driven Architecture Is . . . 3
2. Messages: Events, Commands and Queries . . . 4
3. Kafka Fundamentals . . . 4
4. Choreography vs Orchestration . . . 5
5. The Example Platform . . . 6

Part II — Building It 8

6. Setup . . . 8
7. Event Contracts . . . 9
8. Producing Events . . . 11
9. Consuming Events . . . 12
10. The Transactional Outbox . . . 14
11. Idempotent Consumers . . . 18
12. Orchestration: a Temporal Saga Driven by Kafka . . . 20
13. Choreography and CQRS . . . 25

Part III — Testing and Running 30

14. Testing Strategy . . . 30
15. Running the Platform . . . 37
16. Failure Drills . . . 38

Part IV — Production 39

17. Partitions, Ordering and Scaling . . . 39
18. Schema Evolution . . . 39
19. Consistency Traps . . . 39
20. Observability . . . 40
21. Security and Operations . . . 40
22. Checklist . . . 40

Appendices 41

A. Kafka CLI Cheat Sheet . . . 41
B. Troubleshooting . . . 41
C. Glossary . . . 41

Part I — Concepts

1. What Event-Driven Architecture Is

In a **request-driven** system, services call each other: the order service calls payments, then inventory, then e-mail, and waits for each answer. In an **event-driven** system, services announce facts — *an order was placed*, *a payment succeeded* — and other services react on their own schedule. The producer neither knows nor waits for its consumers.

	Request-driven (sync calls)	Event-driven (messages)
Coupling	Caller knows every callee and its API	Producer knows only the event; consumers come and go
Availability	One slow/down service blocks the chain	Messages wait in Kafka until the consumer is back
Scaling	Every service sized for peak traffic	Consumers scale independently; the log absorbs bursts
New features	Change the caller to call one more service	Add a consumer; nothing else changes
Audit / replay	Must be built separately	The log *is* a history; new consumers can replay it
Consistency	Immediate (inside one call chain)	**Eventual**: views lag slightly behind
Debugging	One stack trace	A flow spans many services: needs correlation IDs and tracing

1.1 What you pay for it

- **Duplicates**: brokers and relays deliver at least once. Every consumer with side effects must be idempotent.
- **Ordering** is per partition, not global.
- **Eventual consistency**: the read model may show PLACED for a moment after payment succeeded. UIs and APIs must be designed for it (202 + status URL).
- **Schema evolution**: events are a public API that outlives the code that wrote them.
- **Long business processes** (wait for payment, compensate on failure) are hard to see when they are spread across many consumers. This is where Temporal comes in.

Don't use EDA for simple CRUD apps or for flows that genuinely need an immediate, consistent answer ("is this username taken?"). Use it where decoupling, buffering, fan-out and auditability pay for the extra moving parts.

2. Messages: Events, Commands and Queries

Kind	Example	Name	Receivers	Expectation
Event	order.placed.v1	Past tense: a fact	0..n, unknown to the producer	None: the producer moves on
Command	payment.requested.v1	Imperative: a request	Exactly one service	Somebody will act and report back
Query	GET /orders/{id}	Question	One service	An immediate answer (usually HTTP, not Kafka)

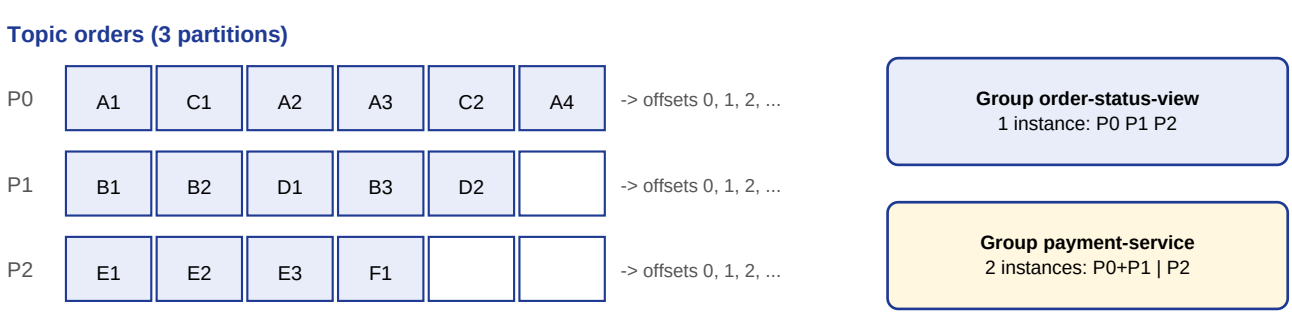
2.1 How much data goes into an event?

Style	What the event contains	Trade-off
Event notification	Just IDs: "order 42 changed"	Small, but consumers call back for details (coupling, load)
Event-carried state transfer	Everything consumers need (email, items, total...)	Bigger events, fully autonomous consumers. **Used in this guide.**
Event sourcing	Every state change; the log is the source of truth	Full history and replay, but a different programming model

2.2 Naming and versioning

- `domain.entity.happened.vN`: `order.placed.v1`, `payment.failed.v1`.
- Additive changes (new optional fields) keep the version; renaming or removing a field or changing its meaning is a new version that coexists on the same topic.
- Validate every incoming message against its schema; reject what you don't understand to a dead-letter topic instead of crashing.

3. Kafka Fundamentals



Same key (A, B, ...) -> same partition -> ordered. Each group reads everything; members of a group split partitions.

Concept	What it means for your design
Topic	A named, append-only log. Here: `orders`, `payments.commands`, `payments.events` (+ a `.dlq` each).
Partition	Unit of parallelism and ordering. Order is guaranteed only within one partition.
Key	Chooses the partition (hash). Same key, same partition, same order. Key by the entity whose events must stay ordered (orderId).
Offset	Position in a partition. Consumers commit offsets; after a restart they resume from the last commit.
Consumer group	One logical subscriber. Every group gets every message; instances in a group split the partitions (max useful instances = partitions).
Retention / compaction	Messages stay for a time/size limit, or, compacted, the latest value per key is kept forever.
Replication / ISR	Each partition is copied to replicas; `acks=all` waits for the in-sync replicas. (Single broker in development.)
KRaft	Kafka's built-in consensus (no ZooKeeper since Kafka 4.0).

3.1 Delivery guarantees

Guarantee	How it happens	Consequence
At most once	Commit the offset, then process	A crash loses the message
At least once	Process, then commit (this guide)	A crash re-delivers: handlers must be idempotent
Exactly once	Kafka transactions for read-process-write *within Kafka*	Doesn't cover side effects outside Kafka (DB, e-mail, HTTP)

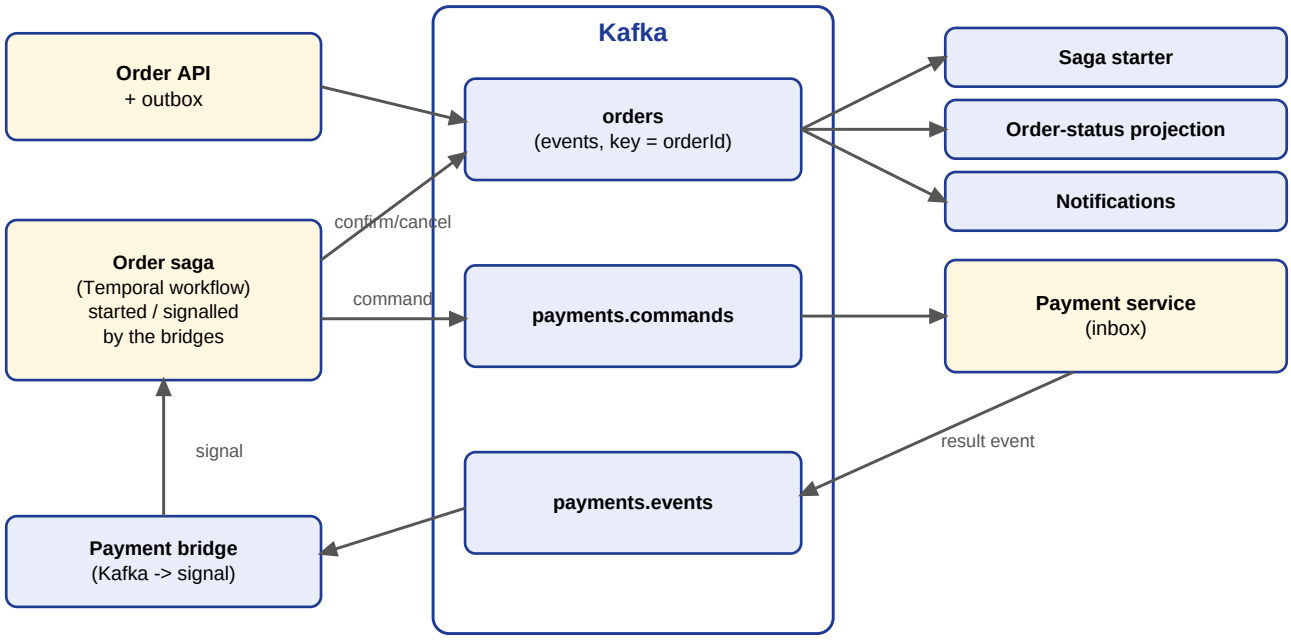
Tip: The practical recipe: **at-least-once delivery + idempotent consumers** (an inbox, deterministic IDs, upserts). The tests in Part III prove it end to end.

4. Choreography vs Orchestration

	Choreography	Orchestration
Who decides the next step	Each service, by reacting to events	One coordinator (here: a Temporal workflow)
Coupling	Lowest: services only know events	Coordinator knows the steps
Visibility of the process	Spread across services and topics	One function; one history per order in the Temporal UI
Timeouts, compensation, retries	Hand-built in every service	Built in: timers, retry policies, sagas
Best for	Fan-out side effects (e-mail, analytics, search, read models)	Multi-step business transactions with deadlines and compensation

This guide uses **both**: the order saga is orchestrated by Temporal (reserve stock → request payment → wait → confirm or compensate), while the read model and notifications simply react to events. Kafka connects everything; Temporal never replaces it — it owns the *process state*, Kafka owns the *event stream*.

5. The Example Platform



5.1 Life of an order

#	Step	Mechanism
1	POST /orders: order row + OrderPlaced in one DB transaction; 202 Accepted	Transactional outbox
2	Relay publishes OrderPlaced to `orders` (key = orderId)	Idempotent producer
3	Saga starter starts workflow `order-saga-{orderId}`	Kafka → Temporal, Workflow ID de-dup
4	Saga reserves stock, publishes PaymentRequested to `payments.commands`	Activity, deterministic event ID
5	Payment service charges, publishes PaymentSucceeded/Failed	Inbox (idempotent consumer)
6	Payment bridge signals the saga	Kafka → Temporal signal
7	Saga publishes OrderConfirmed (or releases stock + OrderCancelled)	Saga compensation
8	Projection updates the read model; notifications e-mail the customer	Choreography, CQRS

5.2 Patterns and where they live

Problem	Pattern	Code
Save + publish atomically	Transactional outbox	`order-api/outbox.ts`
Duplicate deliveries	Inbox / idempotent consumer, deterministic IDs	`inbox.ts`, `saga/activities.ts`
Messages that always fail	Dead-letter topic	`kafka/consumer.ts`
Untrusted payloads	Typed, validated contracts	`events/contracts.ts`
Long-running process	Saga orchestrated by Temporal	`saga/workflows.ts`
Kafka ↔ workflow	Starter + signal bridge	`saga/bridges.ts`
Queries over events	CQRS projection	`projections/order-status.ts`
Tracing a flow	Correlation ID in envelope + headers	everywhere

5.3 Tested with

Everything in Parts II and III was type-checked, run and tested (17 tests) against a real Kafka broker and a real Temporal dev server. The project is `eda-commerce/` (github.com/tcsr/eda-commerce).

Component	Version	Notes
Kafka	4.1.0	Docker image `apache/kafka`, single node, KRaft
Kafka client	@confluentinc/kafka-javascript 1.11	librdkafka 2.16; KafkaJS-compatible API
Temporal	@temporalio/* 1.24	Dev server via `@temporalio/testing`
Fastify / TypeBox	5.12 / 1.3	Order API, event schemas
Node.js / TypeScript	24.15 / 7.0	`node:sqlite` for service databases

Tip: Why the Confluent client? KafkaJS, long the default for Node.js, has not been maintained since 2023. Confluent's client wraps librdkafka (the C library behind most Kafka clients) and offers a KafkaJS-compatible API, so KafkaJS code ports with few changes.

Part II — Building It

6. Setup

```
mkdir eda-commerce && cd eda-commerce && npm init -y
npm install @confluentinc/kafka-javascript fastify @fastify/type-provider-typebox \
            typebox @temporalio/client @temporalio/worker @temporalio/workflow \
            @temporalio/activity @temporalio/common
npm install -D typescript tsx @types/node @temporalio/testing
```

Kafka runs in Docker in KRaft mode: one container acts as both controller and broker. Auto topic creation is off on purpose.

```
// docker-compose.yml
# Single-node Kafka in KRaft mode (no ZooKeeper) for local development and tests
services:
  kafka:
    image: apache/kafka:4.1.0
    container_name: eda-kafka
    ports:
      - "9092:9092"
    environment:
      KAFKA_NODE_ID: 1
      KAFKA_PROCESS_ROLES: broker,controller
      KAFKA_LISTENERS: PLAINTEXT://:9092,CONTROLLER://:9093
      KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://localhost:9092
      KAFKA_CONTROLLER_LISTENER_NAMES: CONTROLLER
      KAFKA_LISTENER_SECURITY_PROTOCOL_MAP: CONTROLLER:PLAINTEXT,PLAINTEXT:PLAINTEXT
      KAFKA_CONTROLLER_QUORUM_VOTERS: 1@localhost:9093
      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
      KAFKA_TRANSACTION_STATE_LOG_REPLICATION_FACTOR: 1
      KAFKA_TRANSACTION_STATE_LOG_MIN_ISR: 1
      KAFKA_GROUP_INITIAL_REBALANCE_DELAY_MS: 0
      KAFKA_AUTO_CREATE_TOPICS_ENABLE: "false"
    healthcheck:
      test: /opt/kafka/bin/kafka-topics.sh --bootstrap-server localhost:9092 --list
      interval: 5s
      timeout: 10s
      retries: 20
```

Verify the broker

```
docker compose up -d --wait     # waits for the healthcheck
npm run smoke                   # create topic, produce 4 messages, consume them back
# sent partitions: [ 1 ]
# received: [ 'order-0@p1:{"i":2}:c2', 'order-0@p1:{"i":4}:c4', ... ]
```

Project layout

```
src/
  config.ts                 topic names (prefixable for tests), broker list
  events/contracts.ts       typed, versioned event envelope + validation
  kafka/client.ts           topics, idempotent producer
  kafka/consumer.ts         consumer runner: at-least-once, retries, DLQ
  inbox.ts                  idempotent-consumer helper
  order-api/                Fastify API + transactional outbox
  payment-service/          command handler with inbox
  saga/                     Temporal workflow, activities, Kafka bridges
  projections/              CQRS read model
  notifications/            choreographed side effect
  inventory.ts              stock reservations (called by the saga)
  platform.ts               wires everything (one process for demo and tests)
test/                       contracts, Kafka basics, reliability, end-to-end
```

7. Event Contracts

Events are an API between teams. They are defined once, typed with TypeBox (so TypeScript knows every field), and validated when consumed.

```
// src/events/contracts.ts
import { Type, Static, TSchema } from 'typebox';
import { Value } from 'typebox/value';
import { randomUUID } from 'node:crypto';

// Every message shares one envelope (modelled on CloudEvents); `data` varies by type  1
const envelope = <T extends string, D extends TSchema>(type: T, data: D) =>
  Type.Object({
    id: Type.String(), // unique per event: the de-duplication key  2
    type: Type.Literal(type), // name + version: 'order.placed.v1'  3
    source: Type.String(), // producing service
    time: Type.String(), // ISO-8601
    subject: Type.String(), // aggregate ID, also the Kafka message key  4
    correlationId: Type.String(), // ties every message of one business flow together  5
    data,
  });
```

1	A common envelope lets every consumer route, log and de-duplicate without knowing the payload.
2	Consumers remember IDs they have processed (the inbox, Chapter 11).
3	Past-tense name + version. A breaking change gets `.v2`; both versions can coexist on the topic.
4	The aggregate (order) ID becomes the key: all events of one order land in one partition, in order.
5	Follow one customer action across API, Kafka, Temporal and e-mails.

```
const Item = Type.Object({
  sku: Type.String(),
  qty: Type.Integer({ minimum: 1 }),
  priceCents: Type.Integer({ minimum: 0 }),
});

export const OrderPlaced = envelope(
  'order.placed.v1',
  Type.Object({
    orderId: Type.String(),
    customerId: Type.String(),
    email: Type.String(),
    items: Type.Array(Item, { minItems: 1 }),
    totalCents: Type.Integer({ minimum: 1 }),
  }),
);
export const OrderConfirmed = envelope(
  'order.confirmed.v1',
  Type.Object({
    orderId: Type.String(),
    email: Type.String(),
    paymentId: Type.String(),
  }),
);
export const OrderCancelled = envelope(
  'order.cancelled.v1',
  Type.Object({
    orderId: Type.String(),
    email: Type.String(),
    reason: Type.String(),
  }),
);
// A command: addressed to one service, asks it to do something  6
export const PaymentRequested = envelope(
  'payment.requested.v1',
  Type.Object({
    orderId: Type.String(),
```

```
    customerId: Type.String(),
    amountCents: Type.Integer({ minimum: 1 }),
  }),
);
export const PaymentSucceeded = envelope(
  'payment.succeeded.v1',
  Type.Object({ orderId: Type.String(), paymentId: Type.String() }),
);
export const PaymentFailed = envelope(
  'payment.failed.v1',
  Type.Object({ orderId: Type.String(), reason: Type.String() }),
);
```

6 Commands (imperative: *RequestPayment*) have one intended receiver; events (past tense: *PaymentSucceeded*) have any number.

```
const SCHEMAS = {
  'order.placed.v1': OrderPlaced,
  'order.confirmed.v1': OrderConfirmed,
  'order.cancelled.v1': OrderCancelled,
  'payment.requested.v1': PaymentRequested,
  'payment.succeeded.v1': PaymentSucceeded,
  'payment.failed.v1': PaymentFailed,
};

export type EventType = keyof typeof SCHEMAS;
export type EventOf<T extends EventType> = Static<(typeof SCHEMAS)[T]>;
export type DomainEvent = { [T in EventType]: EventOf<T> }[EventType];

// A message that can never be processed: retrying is pointless, send it to the DLQ
export class PoisonMessageError extends Error {}  7
```

7 A separate error class lets the consumer skip retries for messages that can never succeed.

```
export function parseEvent(raw: Buffer | null | undefined): DomainEvent {
  let json: { type?: string };
  try {
    json = JSON.parse(String(raw));
  } catch {
    throw new PoisonMessageError('Message is not valid JSON');
  }
  const schema = SCHEMAS[json?.type as EventType];
  if (!schema) throw new PoisonMessageError(`Unknown event type ${json?.type}`);
  if (!Value.Check(schema, json)) {  8
    const first = [...Value.Errors(schema, json)][0];
    throw new PoisonMessageError(
      `${json.type}: ${first?.instancePath} ${first?.message}`,
    );
  }
  return json as DomainEvent;
}
```

8 Validate on the way **in**: producers can be buggy, other teams evolve their code, and the payload is untrusted input.

```ts
export function newEvent<T extends EventType>(
  type: T,
  subject: string,
  data: EventOf<T>['data'],
  meta: { source: string; correlationId: string; id?: string },
): EventOf<T> {
  return {
    id: meta.id ?? randomUUID(),  9
    type,
    source: meta.source,
    time: new Date().toISOString(),
    subject,
    correlationId: meta.correlationId,
    data,
  } as EventOf<T>;
}
```

9 Random by default; a caller can pass a deterministic ID (the saga does).

8. Producing Events

```ts
// src/kafka/client.ts
import { KafkaJS } from '@confluentinc/kafka-javascript';
import { BROKERS, dlqOf } from '../config';
import type { DomainEvent } from '../events/contracts';

export type Kafka = KafkaJS.Kafka;

export function createKafka(clientId: string): Kafka {
  return new KafkaJS.Kafka({
    kafkaJS: { clientId, brokers: BROKERS, logLevel: KafkaJS.logLevel.ERROR },
  });
}

// Topics are created explicitly (auto-create is off):  1
// the partition count is a design decision, not an accident
export async function ensureTopics(kafka: Kafka, topics: string[], partitions = 3) {
  const admin = kafka.admin();
  await admin.connect();
  try {
    const existing = new Set(await admin.listTopics());
    const missing = [...topics, ...topics.map(dlqOf)].filter(  2
      (t) => !existing.has(t),
    );
    if (missing.length) {
      await admin.createTopics({
        topics: missing.map((topic) => ({ topic, numPartitions: partitions })),
      });
    }
  } finally {
    await admin.disconnect();
  }
}
```

1 Auto-created topics get default settings and invite typos; create them deliberately (or with infrastructure-as-code).

2 Every topic gets a dead-letter twin, `orders.dlq`.

```
export interface Publisher {
  publish(topic: string, event: DomainEvent): Promise<void>;
  publishRaw(topic: string, msg: KafkaJS.Message): Promise<void>;
  close(): Promise<void>;
}

export async function createPublisher(kafka: Kafka): Promise<Publisher> {
  // Idempotent producer: broker-side de-duplication of producer retries,
  // acks from all in-sync replicas, ordering preserved per partition
  const producer = kafka.producer({ kafkaJS: { idempotent: true } });  3
  await producer.connect();
  return {
    async publish(topic, event) {
      await producer.send({
        topic,
        messages: [
          {
            key: event.subject, // same key -> same partition -> ordered per order  4
            value: JSON.stringify(event),
            headers: {
              'event-id': event.id,  5
              'event-type': event.type,
              'correlation-id': event.correlationId,
            },
          },
        ],
      });
    },
    async publishRaw(topic, msg) {
      await producer.send({ topic, messages: [msg] });
    },
    close: () => producer.disconnect(),
  };
}
```

3	Idempotent producer: retries after a lost acknowledgement don't write duplicates to the log, and per-partition order is kept.
4	Ordering is guaranteed **per partition** only. Choose the key = the thing whose events must stay in order.
5	Headers: routing and tracing metadata readable without parsing the body (verified with `kafka-console-consumer`).

9. Consuming Events

One runner used by every service: parse and validate, call the handler, retry transient failures a few times, send everything else to the dead-letter topic, then let the offset be committed.

```ts
// src/kafka/consumer.ts
import { setTimeout as delay } from 'node:timers/promises';
import { dlqOf } from '../config';
import { parseEvent, DomainEvent, PoisonMessageError } from '../events/contracts';
import type { Kafka, Publisher } from './client';

export interface MessageContext {
  topic: string;
  partition: number;
  offset: string;
}

export interface ConsumerOptions {
  kafka: Kafka;
  groupId: string; // one group per service: each group gets every message  1
  topics: string[];
  handle: (event: DomainEvent, ctx: MessageContext) => Promise<void>;
  dlq: Publisher; // where unprocessable messages go
  maxAttempts?: number;
  backoffMs?: number;
  log?: (msg: string) => void;
}
```

1 Consumer group = one logical subscriber. Instances in the same group split the partitions; different groups each get everything.

```ts
export interface RunningConsumer {
  stop(): Promise<void>;
}

export async function runConsumer(opts: ConsumerOptions): Promise<RunningConsumer> {
  const { kafka, groupId, topics, handle, dlq } = opts;
  const maxAttempts = opts.maxAttempts ?? 3;
  const backoffMs = opts.backoffMs ?? 200;
  const log = opts.log ?? (() => {});

  const consumer = kafka.consumer({ kafkaJS: { groupId, fromBeginning: true } });  2
  await consumer.connect();
  await consumer.subscribe({ topics });

  await consumer.run({
    // Offsets are committed only after eachMessage resolves: at-least-once delivery  3
    eachMessage: async ({ topic, partition, message }) => {
      const ctx = { topic, partition, offset: message.offset };
      const toDlq = async (err: unknown, attempts: number) => {
        log(`${groupId}: ${topic}@${message.offset} -> DLQ (${String(err)})`);
        await dlq.publishRaw(dlqOf(topic), {  4
          key: message.key,
          value: message.value,
          headers: {
            ...message.headers,
            'dlq-original-topic': topic,
            'dlq-original-offset': message.offset,
            'dlq-consumer-group': groupId,
            'dlq-error': String(err),
            'dlq-attempts': String(attempts),
          },
        });
      };
```

2 `fromBeginning` only matters the first time a group starts; afterwards it resumes from its committed offsets.

3 Crash before the commit → the message is delivered again. Design every handler for duplicates.

4 The original bytes go to the DLQ untouched, plus diagnostics headers for whoever investigates or replays it.

```
      let event: DomainEvent;
      try {
        event = parseEvent(message.value);
      } catch (err) {
        if (err instanceof PoisonMessageError) return toDlq(err, 0); // never retry  5
        throw err;
      }

      // Bounded in-process retries for transient failures, then park in the DLQ  6
      // (retrying in place blocks this partition: keep attempts and backoff small)
      for (let attempt = 1; ; attempt++) {
        try {
          return await handle(event, ctx);
        } catch (err) {
          if (attempt >= maxAttempts) return toDlq(err, attempt);
          log(`${groupId}: attempt ${attempt} failed (${String(err)}), retrying`);
          await delay(backoffMs * 2 ** (attempt - 1));  7
        }
      }
    },
  });

  return { stop: () => consumer.disconnect() };
}
```

5	Poison messages are parked immediately (`dlq-attempts: 0` in the test).
6	Retrying in place keeps order but blocks the partition; for long outages use retry topics or pause the consumer instead.
7	Exponential backoff between attempts.

10. The Transactional Outbox

The order service must store the order *and* tell the world about it. Writing to the database and to Kafka separately can fail half-way: an order nobody hears about, or an event about an order that was rolled back. The outbox makes it one local transaction.

```
// src/order-api/outbox.ts
import type { DatabaseSync } from 'node:sqlite';
import type { Publisher } from '../kafka/client';
import type { DomainEvent } from '../events/contracts';

// Transactional outbox: the event is written in the SAME database transaction as  1
// the business change, then a relay publishes it. No "saved but never published"
// and no "published but rolled back".
export function createOutbox(db: DatabaseSync) {
  db.exec(`CREATE TABLE IF NOT EXISTS outbox (
             seq INTEGER PRIMARY KEY AUTOINCREMENT,
             topic TEXT NOT NULL,
             event TEXT NOT NULL,
             published_at TEXT)`);
  const insert = db.prepare('INSERT INTO outbox (topic, event) VALUES (?, ?)');
  const pending = db.prepare(
    `SELECT seq, topic, event FROM outbox
     WHERE published_at IS NULL ORDER BY seq LIMIT ?`,
  );
  const markPublished = db.prepare(
    'UPDATE outbox SET published_at = ? WHERE seq = ?',
  );
```

1	The dual-write problem: writing to the DB *and* Kafka can't be atomic. The outbox turns it into one local transaction plus a relay.

```
  return {
    // Call inside the caller's transaction
    add(topic: string, event: DomainEvent) {  2
      insert.run(topic, JSON.stringify(event));
    },

    // Publish pending events in order. A crash between publish and mark means the  3
    // event is published again on restart: at-least-once, so consumers de-duplicate.
    async relayOnce(publisher: Publisher, batch = 100): Promise<number> {
      const rows = pending.all(batch) as {
        seq: number;
        topic: string;
        event: string;
      }[];
      for (const row of rows) {
        await publisher.publish(row.topic, JSON.parse(row.event));
        markPublished.run(new Date().toISOString(), row.seq);
      }
      return rows.length;
    },
```

2 Called inside the business transaction (see the API below).

3 The relay is at-least-once too: the test simulates exactly this crash and sees the event twice on the topic.

```
    // Polling relay; production alternatives: CDC with Debezium reading the WAL  4
    start(publisher: Publisher, everyMs = 100) {
      let running = true;
      const loop = async () => {
        while (running) {
          const n = await this.relayOnce(publisher).catch(() => 0);
          if (n === 0) await new Promise((r) => setTimeout(r, everyMs));
        }
      };
      const done = loop();
      return {
        async stop() {
          running = false;
          await done;
        },
      };
    },
  };
}

export type Outbox = ReturnType<typeof createOutbox>;
```

4 Change data capture avoids polling and preserves commit order across instances.

```
// src/order-api/app.ts
import Fastify from 'fastify';
import { Type, TypeBoxTypeProvider } from '@fastify/type-provider-typebox';
import { DatabaseSync } from 'node:sqlite';
import { randomUUID } from 'node:crypto';
import { newEvent } from '../events/contracts';
import { createOutbox } from './outbox';
import type { Topics } from '../config';
import type { OrderViews } from '../projections/order-status';

const OrderBody = Type.Object({
  customerId: Type.String({ minLength: 1 }),
  email: Type.String({ format: 'email' }),
  items: Type.Array(
    Type.Object({
      sku: Type.String(),
      qty: Type.Integer({ minimum: 1 }),
      priceCents: Type.Integer({ minimum: 0 }),
    }),
    { minItems: 1 },
  ),
});
```

```
export function buildOrderApi({
  topics,
  views,
}: {
  topics: Topics;
  views: OrderViews;
}) {
  const db = new DatabaseSync(':memory:');
  db.exec(`CREATE TABLE orders (
             id TEXT PRIMARY KEY, customer_id TEXT,
             total_cents INTEGER, created_at TEXT)`);
  const outbox = createOutbox(db);
  const app = Fastify().withTypeProvider<TypeBoxTypeProvider>();

  app.post('/orders', { schema: { body: OrderBody } }, async (request, reply) => {
    const { customerId, email, items } = request.body;
    const orderId = randomUUID();
    const correlationId = String(
      request.headers['x-correlation-id'] ?? randomUUID(),  1
    );
    const totalCents = items.reduce((s, i) => s + i.qty * i.priceCents, 0);
```

1 Take the caller's correlation ID (or start one): it travels with every message from here on.

```
    // One local transaction: the order row AND the event, or neither
    db.exec('BEGIN');  2
    try {
      db.prepare('INSERT INTO orders VALUES (?, ?, ?, ?)').run(
        orderId,
        customerId,
        totalCents,
        new Date().toISOString(),
      );
      outbox.add(  3
        topics.orders,
        newEvent(
          'order.placed.v1',
          orderId,
          { orderId, customerId, email, items, totalCents },
          { source: 'order-api', correlationId },
        ),
      );
      db.exec('COMMIT');
    } catch (err) {
      db.exec('ROLLBACK');
      throw err;
    }
```

2	One local transaction...
3	...the order row **and** the OrderPlaced event.

```
    // The rest happens asynchronously: answer 202 with where to look
    reply.code(202).header('location', `/orders/${orderId}`);  4
    return { orderId, correlationId };
  });

  // Read side (CQRS): served from the projection built from events  5
  app.get<{ Params: { id: string } }>('/orders/:id', async (request, reply) => {
    const view = views.get(request.params.id);
    if (!view) return reply.code(404).send({ message: 'Unknown order (yet)' });
    return view;
  });

  return { app, db, outbox };
}
```

4	The rest happens asynchronously: 202 Accepted, poll the status URL.
5	Reads come from a separate read model that is **eventually** consistent.

11. Idempotent Consumers

```ts
// src/inbox.ts
import type { DatabaseSync } from 'node:sqlite';

// Inbox / idempotent consumer: Kafka delivers at least once, so every consumer
// that has side effects remembers which event IDs it has already processed.
export function createInbox(db: DatabaseSync, consumer: string) {
  db.exec(`CREATE TABLE IF NOT EXISTS processed_events (
             consumer TEXT NOT NULL,
             event_id TEXT NOT NULL,
             processed_at TEXT NOT NULL,
             PRIMARY KEY (consumer, event_id))`);  1
  const mark = db.prepare(
    'INSERT OR IGNORE INTO processed_events VALUES (?, ?, ?)',  2
  );
```

1 One row per (consumer, event): each service de-duplicates independently.

2 Postgres equivalent: `INSERT ... ON CONFLICT DO NOTHING`.

```ts
  return {
    // Runs `work` only the first time this event ID is seen. The marker and the
    // work's own writes share one transaction: both happen, or neither does.
    async once(
      eventId: string,
      work: () => Promise<void> | void,
    ): Promise<boolean> {
      db.exec('BEGIN IMMEDIATE');  3
      try {
        const fresh = mark.run(consumer, eventId, new Date().toISOString());
        if (fresh.changes === 0) {
          db.exec('ROLLBACK');
          return false; // duplicate delivery: already handled  4
        }
        await work();
        db.exec('COMMIT');
        return true;
      } catch (err) {
        db.exec('ROLLBACK');
        throw err;
      }
    },
  };
}
```

3 The marker and the handler's own writes commit together.

4 Second delivery of the same event: no effect (proven in the reliability test).

The payment service combines the inbox with its own data: a command it has already processed is ignored, so a duplicate can never charge twice.

```ts
// src/payment-service/service.ts
import { DatabaseSync } from 'node:sqlite';
import { randomUUID } from 'node:crypto';
import { newEvent, DomainEvent } from '../events/contracts';
import { createInbox } from '../inbox';
import { runConsumer } from '../kafka/consumer';
import type { Kafka, Publisher } from '../kafka/client';
import type { Topics } from '../config';

export interface PaymentServiceOptions {
  kafka: Kafka;
  publisher: Publisher;
  topics: Topics;
  creditLimitCents?: number; // amounts above this are declined (stands in for a PSP)
}
```

```ts
// Autonomous service: consumes commands, owns its data, publishes facts  (1)
export async function startPaymentService(opts: PaymentServiceOptions) {
  const { kafka, publisher, topics, creditLimitCents = 50_000 } = opts;
  const db = new DatabaseSync(':memory:');
  db.exec(`CREATE TABLE charges (
             payment_id TEXT PRIMARY KEY, order_id TEXT UNIQUE,
             amount_cents INTEGER)`);
  const inbox = createInbox(db, 'payment-service');
  const charge = db.prepare('INSERT INTO charges VALUES (?, ?, ?)');

  const handle = async (event: DomainEvent) => {
    if (event.type !== 'payment.requested.v1') return;
    const { orderId, amountCents } = event.data;
    const meta = { source: 'payment-service', correlationId: event.correlationId };
```

(1) Nobody else reads its tables; others learn about payments only through its events.

```ts
    // Duplicate commands (relay re-sends, activity retries) are ignored here
    await inbox.once(event.id, async () => {  (2)
      const outcome =
        amountCents > creditLimitCents
          ? newEvent(
              'payment.failed.v1',
              orderId,
              { orderId, reason: 'Insufficient funds' },
              meta,
            )
          : newEvent(
              'payment.succeeded.v1',
              orderId,
              { orderId, paymentId: `pay_${randomUUID().slice(0, 8)}` },
              meta,
            );
      if (outcome.type === 'payment.succeeded.v1') {
        charge.run(outcome.data.paymentId, orderId, amountCents);  (3)
      }
      await publisher.publish(topics.paymentEvents, outcome);  (4)
    });
  };
```

(2) Duplicate commands are absorbed: one charge per order (asserted end to end).

(3) The charge record shares the inbox transaction.

(4) Simplification: the publish is inside the DB transaction. Production code would use an outbox here too.

```
  const consumer = await runConsumer({
    kafka,
    groupId: `${topics.prefix}payment-service`,  5
    topics: [topics.paymentCommands],
    handle,
    dlq: publisher,
  });

  return {
    stop: () => consumer.stop(),
    chargesFor: (orderId: string) =>
      (
        db
          .prepare('SELECT COUNT(*) AS n FROM charges WHERE order_id = ?')
          .get(orderId) as {
          n: number;
        }
      ).n,
  };
}
```

5 Scale out by adding instances with the same group ID (up to the partition count).

12. Orchestration: a Temporal Saga Driven by Kafka

The order process has deadlines, a wait for another service and compensation. As choreography it would be spread over several consumers with hand-made timers and state tables. As a Temporal workflow it is one readable function whose state survives crashes.

```
// src/saga/workflows.ts
import {
  proxyActivities,
  defineSignal,
  defineQuery,
  setHandler,
  condition,
  ActivityFailure,
  ApplicationFailure,
} from '@temporalio/workflow';
import type { Duration } from '@temporalio/common';
import type { SagaActivities } from './activities';
import type { EventOf } from '../events/contracts';

const a = proxyActivities<SagaActivities>({
  startToCloseTimeout: '30 seconds',
  retry: { initialInterval: '1 second', maximumAttempts: 5 },
});
```

```ts
export type PaymentOutcome =
  { ok: true; paymentId: string } | { ok: false; reason: string };

// Delivered by the Kafka -> Temporal bridge when the payment service answers
export const paymentResult = defineSignal<[PaymentOutcome]>('paymentResult');  1
export const sagaStatus = defineQuery<string>('sagaStatus');

export interface SagaInput {
  order: EventOf<'order.placed.v1'>['data'];
  correlationId: string;
  paymentTimeout?: Duration;
}

// Orchestration: one durable function owns the order's business process,
// talking to other services through Kafka commands and events
export async function orderSagaWorkflow(input: SagaInput): Promise<string> {
  const { order, correlationId } = input;
  let status = 'RESERVING_STOCK';
  let payment: PaymentOutcome | undefined;
  setHandler(paymentResult, (outcome) => {
    payment ??= outcome; // first answer wins; redelivered events are ignored  2
  });
  setHandler(sagaStatus, () => status);
```

1 The payment answer arrives from Kafka as a **signal** (see the bridge).

2 Signals can also be delivered twice (Kafka redelivery): keep the first.

```ts
  const cancel = async (reason: string) => {
    status = 'CANCELLED';
    await a.publishOrderCancelled(order, reason, correlationId);
    return status;
  };

  // Step 1: reserve stock (synchronous call to the inventory service)  3
  try {
    await a.reserveStock(order.orderId, order.items);
  } catch (err) {
    if (
      err instanceof ActivityFailure &&
      err.cause instanceof ApplicationFailure &&
      err.cause.type === 'OutOfStock'  4
    ) {
      return cancel(err.cause.message);
    }
    throw err;
  }
```

3 Not everything has to go through Kafka: a quick synchronous call is fine inside an activity.

4 A business failure: cancel without asking for payment (verified: no command on the topic).

```
  // Step 2: ask the payment service via Kafka, then wait durably for its answer
  status = 'AWAITING_PAYMENT';
  await a.requestPayment(order, correlationId);  5
  const answered = await condition(  6
    () => payment !== undefined,
    input.paymentTimeout ?? '5 minutes',
  );

  // Step 3: confirm, or compensate the reservation and cancel
  const outcome = answered ? payment! : undefined;
  if (!outcome || !outcome.ok) {
    await a.releaseStock(order.orderId);  7
    return cancel(outcome ? outcome.reason : 'Payment timed out');
  }
  status = 'CONFIRMED';
  await a.publishOrderConfirmed(order, outcome.paymentId, correlationId);
  return status;
}
```

5	Publishes a **command** to Kafka and returns immediately.
6	A durable wait: survives worker restarts. The payment service can be down for minutes (drill in Chapter 16).
7	Compensation: undo the reservation when payment fails or times out.

```
// src/saga/activities.ts
import { ApplicationFailure } from '@temporalio/activity';
import { newEvent, EventOf } from '../events/contracts';
import type { Publisher } from '../kafka/client';
import type { Topics } from '../config';
import type { Inventory } from '../inventory';

type Order = EventOf<'order.placed.v1'>['data'];

export function createSagaActivities(deps: {
  publisher: Publisher;
  topics: Topics;
  inventory: Inventory;
}) {
  const { publisher, topics, inventory } = deps;
  const meta = (correlationId: string, id: string) => ({
    source: 'order-saga',
    correlationId,
    id, // deterministic: a retried activity publishes the SAME event ID  1
  });
```

1	Temporal retries activities; deterministic event IDs let downstream inboxes drop the repeats.

```
  return {
    async reserveStock(orderId: string, items: Order['items']) {
      const shortage = inventory.reserve(orderId, items);
      if (shortage) throw ApplicationFailure.nonRetryable(shortage, 'OutOfStock');  2
    },

    async releaseStock(orderId: string) {
      inventory.release(orderId); // idempotent: releasing twice is a no-op  3
    },

    async requestPayment(order: Order, correlationId: string) {
      await publisher.publish(
        topics.paymentCommands,  4
        newEvent(
          'payment.requested.v1',
          order.orderId,
          {
            orderId: order.orderId,
            customerId: order.customerId,
            amountCents: order.totalCents,
          },
          meta(correlationId, `${order.orderId}:payment-requested`),
        ),
      );
    },
```

2 Non-retryable: out of stock won't fix itself in a second.

3 Every activity is safe to run more than once.

4 Commands go to the payment service's own topic.

```
    async publishOrderConfirmed(
      order: Order,
      paymentId: string,
      correlationId: string,
    ) {
      await publisher.publish(
        topics.orders,
        newEvent(
          'order.confirmed.v1',
          order.orderId,
          { orderId: order.orderId, email: order.email, paymentId },
          meta(correlationId, `${order.orderId}:confirmed`),
        ),
      );
    },

    async publishOrderCancelled(
      order: Order,
      reason: string,
      correlationId: string,
    ) {
      await publisher.publish(
        topics.orders,
        newEvent(
          'order.cancelled.v1',
          order.orderId,
          { orderId: order.orderId, email: order.email, reason },
          meta(correlationId, `${order.orderId}:cancelled`),
        ),
      );
    },
  };
}

export type SagaActivities = ReturnType<typeof createSagaActivities>;
```

12.1 Bridging Kafka and Temporal

Two small consumers connect the event stream to workflows: one **starts** a saga per order, one turns payment results into **signals**.

```
// src/saga/bridges.ts
import {
  Client,
  WorkflowExecutionAlreadyStartedError,
  WorkflowNotFoundError,
} from '@temporalio/client';
import { runConsumer } from '../kafka/consumer';
import { SAGA_TASK_QUEUE, Topics } from '../config';
import { orderSagaWorkflow, paymentResult } from './workflows';
import type { Kafka, Publisher } from '../kafka/client';
import type { Duration } from '@temporalio/common';

export const sagaId = (orderId: string) => `order-saga-${orderId}`;  1

interface BridgeDeps {
  kafka: Kafka;
  publisher: Publisher;
  topics: Topics;
  temporal: Client;
  paymentTimeout?: Duration;
}
```

1 Business ID → Workflow ID: the link between the Kafka world and Temporal.

```
// Kafka -> Temporal (1): every OrderPlaced event starts one saga.
// The Workflow ID is derived from the order, so a redelivered event cannot  2
// start a second saga: Temporal itself is the de-duplication.
export function startSagaStarter(deps: BridgeDeps) {
  const { kafka, publisher, topics, temporal } = deps;
  return runConsumer({
    kafka,
    groupId: `${topics.prefix}order-saga-starter`,
    topics: [topics.orders],
    dlq: publisher,
    handle: async (event) => {
      if (event.type !== 'order.placed.v1') return;
      try {
        await temporal.workflow.start(orderSagaWorkflow, {
          taskQueue: `${topics.prefix}${SAGA_TASK_QUEUE}`,
          workflowId: sagaId(event.data.orderId),
          args: [
            {
              order: event.data,
              correlationId: event.correlationId,
              paymentTimeout: deps.paymentTimeout,
            },
          ],
          workflowIdReusePolicy: 'REJECT_DUPLICATE',  3
          memo: { correlationId: event.correlationId },
        });
      } catch (err) {
        if (!(err instanceof WorkflowExecutionAlreadyStartedError)) throw err;  4
      }
    },
  });
}
```

2 Verified: re-publishing OrderPlaced leaves the saga's run ID unchanged and the order charged once.

3 Even after the saga finished, a late duplicate can't start it again.

4 Already started = success for an idempotent consumer.

```
// Kafka -> Temporal (2): payment facts become signals to the waiting saga  5
export function startPaymentBridge(deps: BridgeDeps) {
  const { kafka, publisher, topics, temporal } = deps;
  return runConsumer({
    kafka,
    groupId: `${topics.prefix}order-saga-payments`,
    topics: [topics.paymentEvents],
    dlq: publisher,
    handle: async (event) => {
      if (
        event.type !== 'payment.succeeded.v1' &&
        event.type !== 'payment.failed.v1'
      ) {
        return;
      }
      const outcome =
        event.type === 'payment.succeeded.v1'
          ? { ok: true as const, paymentId: event.data.paymentId }
          : { ok: false as const, reason: event.data.reason };
      try {
        await temporal.workflow
          .getHandle(sagaId(event.data.orderId))
          .signal(paymentResult, outcome);
      } catch (err) {
        // The saga already finished (e.g. timed out): a late answer is not an error  6
        if (!(err instanceof WorkflowNotFoundError)) throw err;
      }
    },
  });
}
```

5	Request/reply over Kafka, correlated by order ID.
6	A **late** success after a timeout means money was taken for a cancelled order: production systems refund here (Chapter 19).

13. Choreography and CQRS

Not everything needs a coordinator. The read model and notifications react to order events independently, each in its own consumer group.

```
// src/projections/order-status.ts
import type { DomainEvent } from '../events/contracts';

export interface OrderView {
  orderId: string;
  status: 'PLACED' | 'CONFIRMED' | 'CANCELLED';
  totalCents?: number;
  reason?: string;
  paymentId?: string;
  correlationId: string;
  updatedAt: string;
}

// Read model built by folding events.  1
// In production: a table in Postgres, Redis or Elasticsearch.
export function createOrderViews() {
  const views = new Map<string, OrderView>();
```

1	CQRS: the write side publishes events; this read side reshapes them for one query.

```
  return {
    get: (orderId: string) => views.get(orderId),

    // Projection handler: idempotent by construction
```

```
    // (re-applying an event is harmless)
    apply(event: DomainEvent) {
      const now = event.time;
      switch (event.type) {
        case 'order.placed.v1':
          if (!views.has(event.subject)) {  2
            views.set(event.subject, {
              orderId: event.subject,
              status: 'PLACED',
              totalCents: event.data.totalCents,
              correlationId: event.correlationId,
              updatedAt: now,
            });
          }
          break;
        case 'order.confirmed.v1':  3
          views.set(event.subject, {
            ...views.get(event.subject)!,
            status: 'CONFIRMED',
            paymentId: event.data.paymentId,
            updatedAt: now,
          });
          break;
        case 'order.cancelled.v1':
          views.set(event.subject, {
            ...views.get(event.subject)!,
            status: 'CANCELLED',
            reason: event.data.reason,
            updatedAt: now,
          });
          break;
        default:
          break; // other event types are not relevant for this view
      }
    },
  };
}

export type OrderViews = ReturnType<typeof createOrderViews>;
```

2	Idempotent: a redelivered OrderPlaced doesn't reset a CONFIRMED order.
3	Per-key ordering guarantees PLACED is applied before CONFIRMED.

```
// src/notifications/service.ts
import { DatabaseSync } from 'node:sqlite';
import { createInbox } from '../inbox';
import { runConsumer } from '../kafka/consumer';
import type { Kafka, Publisher } from '../kafka/client';
import type { Topics } from '../config';

export interface Mailer {
  send(to: string, template: string, correlationId: string): Promise<void>;
}

// Choreography: reacts to order events on its own; nobody calls it, nobody waits.  1
// Event-carried state transfer: the email address travels in the event, so this  2
// service never has to query the order service.
export async function startNotifications(deps: {
  kafka: Kafka;
  publisher: Publisher;
  topics: Topics;
  mailer: Mailer;
}) {
  const inbox = createInbox(new DatabaseSync(':memory:'), 'notifications');
  return runConsumer({
    kafka: deps.kafka,
    groupId: `${deps.topics.prefix}notifications`,
    topics: [deps.topics.orders],
```

```
    dlq: deps.publisher,
    handle: async (event) => {
      if (event.type === 'order.confirmed.v1') {
        await inbox.once(event.id, () =>  3
          deps.mailer.send(
            event.data.email,
            'order-confirmed',
            event.correlationId,
          ),
        );
      } else if (event.type === 'order.cancelled.v1') {
        await inbox.once(event.id, () =>
          deps.mailer.send(
            event.data.email,
            'order-cancelled',
            event.correlationId,
          ),
        );
      }
    },
  });
}
```

1	Adding this service needed no change anywhere else.
2	Fat events trade a bigger payload for full autonomy: no synchronous lookups.
3	Customers must not get the same e-mail twice.

13.1 Wiring it together

`platform.ts` starts every component in one process for the demo and the tests. In production each is its own deployment; only the configuration (topics, group IDs, task queue) connects them.

```
// src/platform.ts
import { Worker, NativeConnection } from '@temporalio/worker';
import type { Client } from '@temporalio/client';
import type { Duration } from '@temporalio/common';
import { createKafka, createPublisher, ensureTopics } from './kafka/client';
import { runConsumer } from './kafka/consumer';
import { SAGA_TASK_QUEUE, topicsFor } from './config';
import { buildOrderApi } from './order-api/app';
import { createOrderViews } from './projections/order-status';
import { startPaymentService } from './payment-service/service';
import { startNotifications, Mailer } from './notifications/service';
import { startSagaStarter, startPaymentBridge } from './saga/bridges';
import { createSagaActivities } from './saga/activities';
import { createInventory } from './inventory';

export interface PlatformOptions {
  prefix: string; // topic / group prefix (isolates test runs)
  temporal: Client;
  temporalConnection: NativeConnection;
  mailer: Mailer;
  stock: Record<string, number>;
  paymentTimeout?: Duration;
  creditLimitCents?: number;
}

// Wires every service into one process for demos and tests.
// In production each block below is its own deployable service.
export async function startPlatform(opts: PlatformOptions) {
  const topics = topicsFor(opts.prefix);
  const kafka = createKafka(`${opts.prefix}platform`);
  await ensureTopics(kafka, [
    topics.orders,
    topics.paymentCommands,
    topics.paymentEvents,
```

```
  ]);
  const publisher = await createPublisher(kafka);

  // Read side: projection consumer feeding the order-status view
  const views = createOrderViews();
  const projection = await runConsumer({
    kafka,
    groupId: `${topics.prefix}order-status-view`,
    topics: [topics.orders],
    dlq: publisher,
    handle: async (event) => views.apply(event),
  });

  // Order API + outbox relay
  const api = buildOrderApi({ topics, views });
  const relay = api.outbox.start(publisher);

  // Temporal worker running the saga
  const inventory = createInventory(opts.stock);
  const worker = await Worker.create({
    connection: opts.temporalConnection,
    taskQueue: `${topics.prefix}${SAGA_TASK_QUEUE}`,
    workflowsPath: require.resolve('./saga/workflows'),
    activities: createSagaActivities({ publisher, topics, inventory }),
  });
  const workerRun = worker.run();

  const deps = {
    kafka,
    publisher,
    topics,
    temporal: opts.temporal,
    paymentTimeout: opts.paymentTimeout,
  };
  const starter = await startSagaStarter(deps);
  const bridge = await startPaymentBridge(deps);
  let payments = await startPaymentService({
    kafka,
    publisher,
    topics,
    creditLimitCents: opts.creditLimitCents,
  });
  const notifications = await startNotifications({
    kafka,
    publisher,
    topics,
    mailer: opts.mailer,
  });

  return {
    topics,
    kafka,
    publisher,
    views,
    inventory,
    api: api.app,
    outbox: api.outbox,
    get payments() {
      return payments;
    },
    // Failure drills: take the payment service down and bring it back
    async stopPayments() {
      await payments.stop();
    },
    async startPayments() {
      payments = await startPaymentService({
        kafka,
        publisher,
        topics,
        creditLimitCents: opts.creditLimitCents,
```

```
      });
    },
    async stop() {
      await Promise.allSettled([
        relay.stop(),
        projection.stop(),
        starter.stop(),
        bridge.stop(),
        payments.stop(),
        notifications.stop(),
      ]);
      worker.shutdown();
      await workerRun;
      await api.app.close();
      await publisher.close();
    },
  };
}

export type Platform = Awaited<ReturnType<typeof startPlatform>>;
```

Part III — Testing and Running

14. Testing Strategy

Level	File	Needs	Proves
Contract	`contracts.test.ts`	nothing	Validation, poison detection, deterministic IDs
Broker	`kafka-basics.test.ts`	Kafka	Per-key ordering, fan-out across groups, partition sharing, headers
Patterns	`reliability.test.ts`	Kafka	Outbox, relay crash duplicate, inbox, retries, DLQ, poison
End to end	`platform.test.ts`	Kafka + Temporal	Saga paths, compensation, duplicates, outages, timeouts

Each test file uses a unique prefix for topics, consumer groups and the Temporal task queue, so runs never interfere on the shared broker.

```
// test/reliability.test.ts
// Delivery guarantees in practice: outbox, duplicates, inbox, retries, DLQ
import { test, before, after } from 'node:test';
import assert from 'node:assert/strict';
import { DatabaseSync } from 'node:sqlite';
import {
  createKafka,
  createPublisher,
  ensureTopics,
  Publisher,
} from '../src/kafka/client';
import { runConsumer } from '../src/kafka/consumer';
import { createInbox } from '../src/inbox';
import { topicsFor, dlqOf } from '../src/config';
import { buildOrderApi } from '../src/order-api/app';
import { createOrderViews } from '../src/projections/order-status';
import { uniquePrefix, capture, waitFor } from './helpers';
```

```
const prefix = uniquePrefix('reliability');
const topics = topicsFor(prefix);
const kafka = createKafka('reliability-test');
let publisher: Publisher;

before(async () => {
  await ensureTopics(kafka, [topics.orders, topics.paymentCommands]);
  publisher = await createPublisher(kafka);
});
after(() => publisher.close());

const order = {
  customerId: 'c-1',
  email: 'ann@example.com',
  items: [{ sku: 'MUG', qty: 2, priceCents: 1200 }],
};
```

```
test('outbox: the order and its event are saved together, then relayed', async () => {
  const { app, db, outbox } = buildOrderApi({ topics, views: createOrderViews() });
  const res = await app.inject({
    method: 'POST',
    url: '/orders',
    payload: order,
    headers: { 'x-correlation-id': 'corr-outbox' },
  });
  assert.equal(res.statusCode, 202);
  const pending = db
    .prepare('SELECT COUNT(*) AS n FROM outbox WHERE published_at IS NULL')
    .get() as { n: number };
  assert.equal(pending.n, 1, 'nothing is published inside the request');

  const seen = await capture(kafka, topics.orders, `${prefix}outbox-reader`);
  assert.equal(await outbox.relayOnce(publisher), 1);
  assert.equal(await outbox.relayOnce(publisher), 0, 'already published');
  const msg = await waitFor(() => seen.messages[0], { what: 'relayed event' });
  assert.equal(msg.headers['correlation-id'], 'corr-outbox');
  assert.equal(msg.key, res.json().orderId);
  await seen.stop();
  await app.close();
});
```

```
test('a relay crash re-publishes; an inbox makes the consumer idempotent', async () => {
  const { app, db, outbox } = buildOrderApi({ topics, views: createOrderViews() });
  const { orderId } = (
    await app.inject({ method: 'POST', url: '/orders', payload: order })
  ).json();
  await outbox.relayOnce(publisher);
  // Simulate: published, but the process died before marking the row as sent
  db.exec('UPDATE outbox SET published_at = NULL');  1
  await outbox.relayOnce(publisher);

  const raw = await capture(kafka, topics.orders, `${prefix}raw-${orderId}`);
  await waitFor(() => raw.messages.filter((m) => m.key === orderId).length === 2, {  2
    what: 'the duplicate',
  });
  await raw.stop();
```

1 Simulates the crash between "published" and "marked as published".

2 The duplicate really is on the topic...

```
  const inbox = createInbox(new DatabaseSync(':memory:'), 'loyalty-points');
  const awarded: string[] = [];
  const consumer = await runConsumer({
    kafka,
    groupId: `${prefix}loyalty-${orderId}`,
    topics: [topics.orders],
    dlq: publisher,
    handle: async (event) => {
      if (event.type === 'order.placed.v1' && event.subject === orderId) {
        await inbox.once(event.id, () => {
          awarded.push(event.subject);
        });
      }
    },
  });
  await waitFor(() => awarded.length > 0, { what: 'processing' });
  await new Promise((r) => setTimeout(r, 1500)); // give the duplicate time to arrive
  await consumer.stop();
  assert.deepEqual(awarded, [orderId], 'two deliveries, one effect');  3
  await app.close();
});
```

3 ...and the inbox turns two deliveries into one effect.

```
test('transient failures are retried in place, then succeed', async () => {
  let calls = 0;
  const done: string[] = [];
  const consumer = await runConsumer({
    kafka,
    groupId: `${prefix}flaky`,
    topics: [topics.paymentCommands],
    dlq: publisher,
    backoffMs: 50,
    handle: async (event) => {
      if (event.subject !== 'flaky-1') return;
      if (++calls < 3) throw new Error('downstream timeout');
      done.push(event.subject);
    },
  });
  await publisher.publish(topics.paymentCommands, paymentRequested('flaky-1'));
  await waitFor(() => done.length === 1, { what: 'success after retries' });
  await consumer.stop();
  assert.equal(calls, 3);
});
```

```
test('a permanently failing message goes to the DLQ with diagnostics', async () => {
  const dlq = await capture(
    kafka,
    dlqOf(topics.paymentCommands),
    `${prefix}dlq-reader`,
  );
  const consumer = await runConsumer({
    kafka,
    groupId: `${prefix}broken`,
    topics: [topics.paymentCommands],
    dlq: publisher,
    maxAttempts: 3,
    backoffMs: 20,
    handle: async (event) => {
      if (event.subject === 'bad-1') throw new Error('card network unreachable');
    },
  });
  await publisher.publish(topics.paymentCommands, paymentRequested('bad-1'));
  const parked = await waitFor(() => dlq.messages.find((m) => m.key === 'bad-1'), {
    what: 'DLQ message',
  });
  assert.equal(parked.headers['dlq-attempts'], '3');
  assert.match(parked.headers['dlq-error'], /card network unreachable/);
  assert.equal(parked.headers['dlq-original-topic'], topics.paymentCommands);
  assert.equal(parked.headers['dlq-consumer-group'], `${prefix}broken`);
```

```
  // Poison message: not even valid JSON -> DLQ immediately, no retries
  await publisher.publishRaw(topics.paymentCommands, {
    key: 'poison-1',
    value: '{oops',
  });
  const poison = await waitFor(
    () => dlq.messages.find((m) => m.key === 'poison-1'),
    { what: 'poison in DLQ' },
  );
  assert.equal(poison.headers['dlq-attempts'], '0');
  await consumer.stop();
  await dlq.stop();
});

function paymentRequested(orderId: string) {
  return {
    id: `${orderId}:payment-requested`,
    type: 'payment.requested.v1' as const,
    source: 'test',
    time: new Date().toISOString(),
    subject: orderId,
    correlationId: orderId,
    data: { orderId, customerId: 'c-1', amountCents: 100 },
  };
}
```

```
// test/platform.test.ts
// End to end: HTTP -> outbox -> Kafka -> Temporal saga -> Kafka -> services
//             -> read model, against real Kafka and a real Temporal server
import { test, before, after } from 'node:test';
import assert from 'node:assert/strict';
import { setTimeout as delay } from 'node:timers/promises';
import { TestWorkflowEnvironment } from '@temporalio/testing';
import { startPlatform, Platform } from '../src/platform';
import { sagaId } from '../src/saga/bridges';
import { sagaStatus } from '../src/saga/workflows';
import { uniquePrefix, capture, waitFor } from './helpers';

let env: TestWorkflowEnvironment;
let platform: Platform;
const mails: { to: string; template: string; correlationId: string }[] = [];
```

```
before(async () => {
  env = await TestWorkflowEnvironment.createLocal(); // real Temporal dev server  1
  platform = await startPlatform({
    prefix: uniquePrefix('platform'),  2
    temporal: env.client,
    temporalConnection: env.nativeConnection,
    mailer: {
      send: async (to, template, correlationId) => {
        mails.push({ to, template, correlationId });
      },
    },
    stock: { MUG: 10, LAMP: 1 },
    creditLimitCents: 50_000,
    paymentTimeout: '20 seconds',
  });
});
after(async () => {
  await platform?.stop();
  await env?.teardown();
});
```

1	Real Temporal dev server (not time-skipping): the saga waits on Kafka traffic in real time.
2	Unique topic, group and task-queue names isolate each test run on the shared broker.

```
async function placeOrder(
  items: { sku: string; qty: number; priceCents: number }[],
  email = 'ann@example.com',
) {
  const res = await platform.api.inject({
    method: 'POST',
    url: '/orders',
    payload: { customerId: 'c-1', email, items },
  });
  assert.equal(res.statusCode, 202);
  return res.json() as { orderId: string; correlationId: string };
}

const finalView = (orderId: string, on: Platform = platform) =>  3
  waitFor(
    async () => {
      const res = await on.api.inject(`/orders/${orderId}`);
      const view = res.statusCode === 200 ? res.json() : undefined;
      return view && view.status !== 'PLACED' ? view : undefined;
    },
    { what: `final status of ${orderId}`, timeoutMs: 40_000 },
  );
```

3 Eventual consistency in tests: poll the read model until it settles.

```
test('happy path: reserved, paid, confirmed, customer notified', async () => {
  const { orderId, correlationId } = await placeOrder([
    { sku: 'MUG', qty: 2, priceCents: 1200 },
  ]);
  const view = await finalView(orderId);
  assert.equal(view.status, 'CONFIRMED');
  assert.match(view.paymentId, /^pay_/);
  assert.equal(platform.inventory.available('MUG'), 8);
  assert.equal(platform.payments.chargesFor(orderId), 1);  4
  const mail = await waitFor(
    () => mails.find((m) => m.correlationId === correlationId),
    { what: 'confirmation mail' },
  );
  assert.equal(mail.template, 'order-confirmed');
  const saga = env.client.workflow.getHandle(sagaId(orderId));
  assert.equal(await saga.query(sagaStatus), 'CONFIRMED');
});
```

4 End-to-end proof that the customer was charged exactly once.

```
test('payment declined: the saga releases the stock and cancels', async () => {
  const { orderId } = await placeOrder(
    [{ sku: 'MUG', qty: 1, priceCents: 60_000 }],
    'bob@example.com',
  );
  const view = await finalView(orderId);
  assert.deepEqual([view.status, view.reason], ['CANCELLED', 'Insufficient funds']);
  assert.equal(platform.inventory.available('MUG'), 8, 'reservation compensated');
  await waitFor(
    () =>
      mails.find(
        (m) => m.to === 'bob@example.com' && m.template === 'order-cancelled',
      ),
    { what: 'cancellation mail' },
  );
});
```

```
test('out of stock: cancelled before any payment is requested', async () => {
  const commands = await capture(
    platform.kafka,
    platform.topics.paymentCommands,
    `${platform.topics.prefix}cmd-spy`,
  );
  const { orderId } = await placeOrder([{ sku: 'LAMP', qty: 2, priceCents: 4500 }]);
  const view = await finalView(orderId);
  assert.deepEqual([view.status, view.reason], ['CANCELLED', 'Out of stock: LAMP']);
  await delay(1000);
  assert.equal(commands.messages.filter((m) => m.key === orderId).length, 0);
  await commands.stop();
});

test('a duplicate OrderPlaced event does not start a second saga', async () => {
  const { orderId } = await placeOrder([{ sku: 'MUG', qty: 1, priceCents: 900 }]);
  await finalView(orderId);
  const saga = env.client.workflow.getHandle(sagaId(orderId));
  const runId = (await saga.describe()).runId;
```

```
  // Re-deliver the exact same event, as a crashed outbox relay would
  const original = await captureOrderPlaced(orderId);
  await platform.publisher.publish(
    platform.topics.orders,
    JSON.parse(original.value),
  );
  await delay(3000);

  assert.equal((await saga.describe()).runId, runId, 'same saga run');
  assert.equal(platform.payments.chargesFor(orderId), 1, 'charged once');
});

test('payment service down: the command waits, the saga waits durably', async () => {
  await platform.stopPayments();
  const { orderId } = await placeOrder([{ sku: 'MUG', qty: 1, priceCents: 1500 }]);
  const saga = env.client.workflow.getHandle(sagaId(orderId));
  await waitFor(
    async () =>
      (await saga.query(sagaStatus).catch(() => '')) === 'AWAITING_PAYMENT',
    { what: 'saga waiting for payment' },
  );
  await delay(2000); // still nobody to answer
  assert.equal(await saga.query(sagaStatus), 'AWAITING_PAYMENT');
```

```
  await platform.startPayments(); // service comes back
  const view = await finalView(orderId);
  assert.equal(view.status, 'CONFIRMED');
});

test('no answer before the deadline: the saga times out, compensates', async () => {
  const fast = await startPlatform({
    prefix: uniquePrefix('timeout'),
    temporal: env.client,
    temporalConnection: env.nativeConnection,
    mailer: { send: async () => {} },
    stock: { MUG: 5 },
    paymentTimeout: '3 seconds',
  });
  try {
    await fast.stopPayments();
    const res = await fast.api.inject({
      method: 'POST',
      url: '/orders',
      payload: {
        customerId: 'c-2',
        email: 'cy@example.com',
        items: [{ sku: 'MUG', qty: 1, priceCents: 1000 }],
      },
    });
    const view = await finalView(res.json().orderId, fast);
    assert.deepEqual(
      [view.status, view.reason],
      ['CANCELLED', 'Payment timed out'],
    );
    assert.equal(fast.inventory.available('MUG'), 5, 'reservation released');
  } finally {
    await fast.stop();
  }
});
```

```
async function captureOrderPlaced(orderId: string) {
  const spy = await capture(
    platform.kafka,
    platform.topics.orders,
    `${platform.topics.prefix}spy-${orderId}`,
  );
  const msg = await waitFor(
    () =>
      spy.messages.find(
        (m) => m.key === orderId && m.headers['event-type'] === 'order.placed.v1',
      ),
    { what: 'original OrderPlaced' },
  );
  await spy.stop();
  return msg;
}
```

Result

```
npm test
# ok  a valid event round-trips through JSON
# ok  garbage, unknown types and invalid payloads are poison messages
# ok  deterministic IDs make retried publications recognisable
# ok  same key, same partition: events of one order stay in order
# ok  every consumer GROUP gets every event (pub/sub fan-out)
# ok  consumers in the SAME group share the partitions (scale-out)
# ok  headers carry metadata without touching the payload
# ok  outbox: the order and its event are saved together, then relayed
# ok  a relay crash re-publishes; an inbox makes the consumer idempotent
# ok  transient failures are retried in place, then succeed
# ok  a permanently failing message goes to the DLQ with diagnostics
# ok  happy path: reserved, paid, confirmed, customer notified
# ok  payment declined: the saga releases the stock and cancels
# ok  out of stock: cancelled before any payment is requested
# ok  a duplicate OrderPlaced event does not start a second saga
# ok  payment service down: the command waits, the saga waits durably
# ok  no answer before the deadline: the saga times out, compensates
# tests 17, pass 17, fail 0   (about 40 s)
```

15. Running the Platform

```
npm run kafka:up        # terminal 1: Kafka in Docker
npm run dev-server      # terminal 2: Temporal dev server, UI http://localhost:8233
npm start               # terminal 3: every service, API on http://localhost:3000
```

Real responses from the verification run

```
curl -i -X POST localhost:3000/orders -H "Content-Type: application/json" \
     -H "x-correlation-id: demo-corr-001" \
     -d '{"customerId":"c-42","email":"ann@example.com",
          "items":[{"sku":"MUG","qty":2,"priceCents":1200}]}'
# HTTP/1.1 202 Accepted
# location: /orders/919aed8c-6571-4013-842d-a0dd0a371238
# {"orderId":"919aed8c-...","correlationId":"demo-corr-001"}

curl localhost:3000/orders/919aed8c-6571-4013-842d-a0dd0a371238
# {"status":"CONFIRMED","totalCents":2400,"correlationId":"demo-corr-001",
#  "paymentId":"pay_782bbae1", ...}

# An order above the credit limit
# {"status":"CANCELLED","totalCents":89900,"reason":"Insufficient funds", ...}

# Platform log
# [mail] order-confirmed -> ann@example.com (correlation demo-corr-001)
# [mail] order-cancelled -> bob@example.com (correlation 8791dd67-...)
```

15.1 Looking inside Kafka

```
docker exec eda-kafka /opt/kafka/bin/kafka-console-consumer.sh \
  --bootstrap-server localhost:9092 --topic orders --from-beginning \
  --property print.key=true --property print.headers=true --property print.partition=true
# Partition:0  event-id:080f9dc8-...,event-type:order.placed.v1,
#              correlation-id:demo-corr-001   919aed8c-...   {"id":"080f9dc8-...", ...}
# Partition:0  event-id:919aed8c-...:confirmed,event-type:order.confirmed.v1,
#              correlation-id:demo-corr-001   919aed8c-...   {"id":"919aed8c-...:confirmed", ...}

# payments.commands: "type":"payment.requested.v1" "amountCents":2400
# payments.events:   "type":"payment.succeeded.v1" "paymentId":"pay_782bbae1"

docker exec eda-kafka /opt/kafka/bin/kafka-consumer-groups.sh \
  --bootstrap-server localhost:9092 --describe --all-groups
# notifications        orders             lag=0
# order-saga-payments  payments.events    lag=0
# order-saga-starter   orders             lag=0
# order-status-view    orders             lag=0
# payment-service      payments.commands  lag=0
```

Both events of the order share key and partition 0 (ordered); the saga's event ID is deterministic (`{orderId}:confirmed`); the correlation ID set by the HTTP caller appears on every message. Five consumer groups read independently, all caught up (lag 0).

Watch out: On Windows Git Bash, prefix `docker exec` commands with `MSYS_NO_PATHCONV=1`: otherwise `/opt/kafka/...` is rewritten to a Windows path and the command silently prints nothing (happened during verification).

16. Failure Drills

Each drill is an automated test against the real broker and Temporal server:

Drill	What happened (verified)
Relay crashes after publishing, before marking	OrderPlaced is on the topic **twice**; the inbox-protected consumer applied it **once**
The same OrderPlaced is delivered again later	Saga run ID unchanged; customer charged once
Handler fails twice, then works	3 attempts with backoff; processed; no DLQ entry
Handler always fails	After 3 attempts: DLQ message with original key/bytes and `dlq-error`, `dlq-attempts: 3`, topic, group
Invalid JSON on the topic	Straight to the DLQ (`dlq-attempts: 0`), the partition keeps moving
Payment declined	Stock released (8 → 8), CANCELLED "Insufficient funds", cancellation e-mail
Out of stock	CANCELLED before any payment command was published
Payment service down	Saga stays AWAITING_PAYMENT; command waits in Kafka; service restarts → CONFIRMED
No answer within the deadline (3 s)	Saga times out, releases stock, CANCELLED "Payment timed out"

Part IV — Production

17. Partitions, Ordering and Scaling

- **Partition count** caps parallelism per consumer group. Start with a number that covers expected peak consumers (e.g. 12 or 24); increasing later reshuffles keys across partitions, so per-key ordering breaks for a while.
- **Key choice**: the entity whose events must stay ordered (order, account). Avoid hot keys (one tenant producing most traffic).
- **Rebalances** pause consumption while partitions move between instances. Use cooperative/incremental rebalancing and keep handlers short.
- **Consumer lag** (`kafka-consumer-groups.sh --describe`, or metrics exporters) is the key health signal: alert on growing lag, not on a fixed number.
- **Blocking retries** (this guide) keep order but stall a partition. For long outages, pause the consumer, or move failed messages to delayed retry topics (`orders.retry.1m`, `orders.retry.10m`) and accept reordering for those.

18. Schema Evolution

Change	Safe?	How
Add an optional field	Yes	Same version; old consumers ignore it
Add a required field	No	Add it as optional first, or publish `.v2`
Rename / remove / change meaning	No	New version; publish both during the migration; retire v1 when no consumer needs it
New event type	Yes	Unknown types go to the DLQ in strict consumers: decide whether to ignore instead

At scale, use a **schema registry** (Confluent Schema Registry, Apicurio) with JSON Schema, Avro or Protobuf: producers register schemas, the registry enforces compatibility rules, and consumers fetch the schema by ID from the message.

19. Consistency Traps

- **Late answers**: if the payment succeeds *after* the saga timed out, money was taken for a cancelled order. Handle it: the bridge publishes a refund command when it signals a finished saga, or the payment service checks a deadline carried in the command.
- **Outbox on every producer**: the payment service here publishes inside its database transaction for brevity; a crash between charge and publish would leave the saga waiting until its timeout. Production code uses an outbox there too.
- **Exactly-once** in Kafka (transactions, `isolation.level=read_committed`) covers Kafka-to-Kafka processing only. Side effects in databases and other systems still need idempotency.
- **Read-your-writes**: right after POST the read model may not have the order. Return 202 + Location, or read from the write side for the creating user.

20. Observability

- **Correlation IDs** in the envelope and headers (this guide); log them in every service.
- **Distributed tracing**: propagate W3C `traceparent` in Kafka headers (OpenTelemetry has Kafka instrumentation); Temporal has an OpenTelemetry interceptor.
- **Metrics**: consumer lag per group, DLQ message rate, handler latency and errors, outbox backlog (unpublished rows), saga durations and failure reasons.
- **Temporal UI**: every order's saga history (search by Workflow ID `order-saga-{orderId}`).
- **DLQ tooling**: a small CLI or admin page to inspect, fix and replay DLQ messages back to the original topic.

21. Security and Operations

- TLS for client connections, SASL (SCRAM or OAUTHBEARER) for authentication, ACLs per service (who may produce/consume which topic).
- No secrets or unnecessary personal data in events: they are copied to every consumer and kept for the retention period. Prefer IDs over raw personal data where you can.
- Replication factor 3 and `min.insync.replicas=2` in production; `acks=all` producers (the idempotent producer implies it).
- Retention: days for operational topics; compaction for "latest state per key" topics; DLQs long enough to investigate.
- Each service and each consumer group is its own deployment; scale consumers up to the partition count.

22. Checklist

Item	In this guide
Typed, versioned event contracts, validated on consume	Chapter 7
Keys chosen for ordering; topics created explicitly	Chapter 8
At-least-once consumers with bounded retries + DLQ	Chapter 9
Outbox for every service that changes data and publishes	Chapter 10
Idempotent consumers (inbox, deterministic IDs, upserts)	Chapters 11-12
Long-running processes in a workflow engine	Chapter 12
Read models for queries; 202 + status URL for writes	Chapters 10, 13
Correlation IDs end to end	Chapters 7, 15
Failure drills automated in CI	Chapter 16

Appendices

A. Kafka CLI Cheat Sheet

```
K=/opt/kafka/bin   # inside the container: docker exec -it eda-kafka bash
$K/kafka-topics.sh --bootstrap-server localhost:9092 --list
$K/kafka-topics.sh --bootstrap-server localhost:9092 --describe --topic orders
$K/kafka-topics.sh --bootstrap-server localhost:9092 --create --topic orders \
                   --partitions 12 --replication-factor 3
$K/kafka-console-consumer.sh --bootstrap-server localhost:9092 --topic orders \
                   --from-beginning --property print.key=true --property print.headers=true
$K/kafka-console-producer.sh --bootstrap-server localhost:9092 --topic orders \
                   --property parse.key=true --property key.separator=:
$K/kafka-consumer-groups.sh --bootstrap-server localhost:9092 --describe --all-groups
$K/kafka-consumer-groups.sh --bootstrap-server localhost:9092 --group payment-service \
                   --topic payments.commands --reset-offsets --to-earliest --execute
```

B. Troubleshooting

Symptom	Cause / fix
Consumer receives nothing at first	Group joining / rebalancing takes a few seconds; `fromBeginning` applies only to new groups
Old messages are not re-read	The group has committed offsets. Use a new group or `--reset-offsets`
Topic not found	Auto-create is off: create topics (`ensureTopics`)
One partition stops moving	A handler keeps failing (blocking retry). Check the logs and the DLQ
Duplicates downstream	Expected with at-least-once: add an inbox / deterministic IDs
Temporal bundling: "Failed to load native binding"	Pin `"overrides": { "@swc/core": "1.16.2" }` (see the Temporal tutorial)
`docker exec ... /opt/kafka/...` prints nothing on Windows	Set `MSYS_NO_PATHCONV=1` in Git Bash

C. Glossary

Term	Definition
Event	An immutable fact that something happened, named in the past tense
Command	A request to one service to do something
Topic / partition / offset	Named log / ordered shard of it / position in a shard
Consumer group	Set of consumer instances sharing a subscription
Consumer lag	How far a group is behind the end of the log
DLQ	Dead-letter topic for messages that could not be processed
Outbox	Table written with business data; relayed to the broker
Inbox	Table of processed message IDs used to drop duplicates
Saga	Multi-step process with compensating actions instead of a distributed transaction
Choreography / orchestration	Services react to events / a coordinator directs the steps

Term	Definition
CQRS	Separate models for writes (commands) and reads (projections)
Idempotence	Doing something twice has the same effect as doing it once

Further reading

- Companion guides: Fastify with Node.js, Temporal with Node.js, Fastify + Temporal
- kafka.apache.org/documentation · github.com/confluentinc/confluent-kafka-javascript
- docs.temporal.io/develop/typescript · cloudevents.io